<Nimo> <Al-Azzawi>

Term Project: Instant Messaging App

Architecture Document

Table of Contents

1 Introduction....2

1.1 Purpose....2

1.2 Target Audience....2

1.3 Terms and Definitions....2

2 Architectural Goals and Constraints....3

2.1 Security....3

2.2 Persistency....3

2.3 Reliability/Availability....3

3 System Overview....4

4 System Architecture....5

4.1 Front-end Architecture....5

4.2 Back-end Microservices Architecture....5

5 Database Design....6

6 Issue and Concerns....7

1 Introduction

This document describes the software architecture for the frontend of the Instant Messaging Application, a web-based (Chad Messenger) real-time chat client inspired by classic platforms such as Yahoo Messenger and MSN. It captures the structural decisions, component organisation, data flow, and integration strategy that guide implementation of the React-based client.

1.1 Purpose

The purpose of this document is to provide a clear, structured reference for the frontend architecture of Chad Messenger. It communicates design decisions, components and tools used to develop the front-end like react, node.js, etc.The main targets of this document are anyone who works on or reviews the project, including the greatest TA Huan Wu, and future developers and classmates, course instructor. The document also demonstrates how architectural principles from the course layered architecture, separation of concerns, component-based design are applied in a practical context.

1.2 Target Audience

This document is intended, but does not exclude anyone else:

- TA Huan Wu and The course instructor who will evaluate the architectural decisions made for the frontend.
- future developers who takes over, extends, or maintains the frontend codebase.
- Classmates who want to see an example on how front-end Architecture Document should be documented
- The student developer (Nimo Al-Azzawi), as a reference during implementation

1.3 Terms and Definitions

Term	Definition
React	A JavaScript library for building component-based user interfaces

Component	A self-contained, reusable UI building block in React
JSX	JavaScript XML React's syntax extension for describing UI structure
React Router	A client-side routing library for React single-page applications
Context API	React’s built-in mechanism for sharing state across components without prop-drilling
Socket.io-client	JavaScript library that enables WebSocket-based real time communication from the browser
REST API	Representational State Transfer HTTP-based interface used for non-real-time operations
SPA	Single-Page Application a web app that loads once and updates the view dynamically
JWT	JSON Web Token a compact, self-contained token used for authentication
MERN	MongoDB, Express, React, Node.js the technology stack used for the project
TA	Teaching Assistant provides the complementary backend component
Serv	Any Cloud-based software that offer a service that is rented rather than owned is fundamental to cloud computing.
HMR	Hot Module Replacement

Table 1 - Terms and Definitions

2 Architectural Goals and Constraints

The Chad Messenger front-end is designed around SPA architecture. This architecture prioritises simplicity which helps visualize what are the core component of the chat app. The only Constraint is backend API, since only the front-end is being developed, if the API is not implemented correctly into our system it might break it.

2.1 Security

The front-end enforces security at the presentation layer through the following mechanisms:

- JWT is stored in an HTTP-only cookie set by the backend on login. The browser automatically attaches this cookie to every subsequent request, so no manual token management is required in the front-end code.
- Protected routes implemented with a ProtectedRoute higher-order component prevent unauthenticated users from accessing chat screens; they are redirected to the login page.
- Password fields use the native HTML type="password" attribute to prevent plaintext exposure on screen.
- No sensitive credentials are stored in component state or committed to source control; environment variables are used for API base URLs.

2.2 Persistency

Data persistence is handled entirely by the backend and its MongoDB database. The frontend does not own any persistent data. Its responsibilities related to persistence are:

- Requesting and displaying conversation history from the backend when a user opens a chat room.
- JWT is stored in an HTTP-only cookie set by the backend on login. The browser automatically attaches this cookie to every subsequent request, so no manual token management is required in the front-end code.

- Socket.io client state (active room, message buffer) is held in React Context and is intentionally ephemeral — it is re-hydrated from the server on reconnection.

2.3 Reliability/Availability

The frontend handles network and connection issues by:

- Connection errors from Socket.io are caught and displayed to the user via a persistent error banner component rather than silently failing.
- Axios HTTP errors are intercepted globally and mapped to human-readable messages rendered in the relevant form or panel.
- The application does not depend on any third-party SaaS services; it communicates only with the TA backend, so availability is constrained by that server's uptime.

Constraint: Because real-time messaging depends on a persistent WebSocket connection, temporary network interruptions will cause message delivery to pause. Automatic reconnection is handled by the Socket.io-client library, but messages sent during an outage will be lost unless the backend implements a message queue that holds the messages while offline.

3 System Overview

Chad Messenger is a web-based chat platform made up of two independently deployed components: a React frontend and a Node.js/Express/MongoDB backend which is going to be provided by the TA. The two components communicate over a stable network: the front-end calls REST endpoints for stateful operations (authentication, room management, profile) and maintains a persistent Socket.io WebSocket connection for real-time message delivery.

From the user's perspective, the application presents a single-page interface divided into three regions: a top navigation/profile bar, a left-hand contacts sidebar, and a main direct-message chat window. All navigation between conversations happens without a full page reload, providing a responsive, desktop-app feel consistent with the of the 1990s aesthetic

Presentation Layer	React Components | JSX | CSS Modules
State Management	React Context API | useState / useReducer | Socket.io Client State
Routing Layer	React Router v6 | Protected Routes | Navigation Guards
API Layer	REST API Service | WebSocket Service (Socket.io) | Auth Service
Infrastructure Layer	Axios HTTP Client | Socket.io-client

Figure 3.1- summary of the architectural structure of the overall system.

The Presentation Layer contains all visible React components. The State Management Layer maintains application state using React Context and hooks. The Routing Layer controls which views are rendered based on the URL and authentication status. The Service/API Layer encapsulates all outbound communication logic. The Infrastructure

Layer contains the third-party clients that physically transport data to and from the backend.

4 System Architecture

This section describes the frontend architecture in detail using Presentation Layer to break down the component, and layered component model, and provides a high-level overview of the backend microservices design as context

4.1 Front-end Architecture

The frontend is a React Single-Page Application built with Vite as the development and build tool. All application logic runs in the browser; the server is used only for API calls and WebSocket events. The application is structured according to a five-layer model and follows the component-based design pattern central to React development.

4.1.1 Framework and Build Tooling

React 18 is the UI library. Vite provides fast HMR during development and produces an optimised static bundle for deployment. React Router v6 manages client-side routing. No CSS framework is used; styles are written in plain CSS modules to preserve the nostalgic aesthetic without the overhead of a utility-class system.

4.1.2 Presentation Layer

All UI is expressed as functional React components. The component tree is organised into feature-based folders under src/components/

Component	Description	Key Files
AuthPage	Login and registration forms using email and password, with profile setup flow and error display	AuthPage.jsx, authService.js
ChatLayout	Root layout wrapper: sidebar + main chat area side-by-side	ChatLayout.jsx
ContactSidebar	Scrollable contacts list sorted by last message time; search contacts; open/delete DM conversations + create new Room chat	ContactSidebar.jsx, contactService.js

ChatWindow	Message display area with auto-scroll and timestamp rendering	ChatWindow.jsx
MessageInput	Controlled text input with send button and Enter key support	MessageInput.jsx
SocketProvider	Context provider that initialises and exposes socket.io client	SocketContext.jsx, socketService.js
ProfilePanel	Edit first/last name form, persists via API call	ProfilePanel.jsx
ProtectedRoute	HOC that redirects unauthenticated users to login page	ProtectedRoute.jsx

Table - 2 summarises each major component

Components are kept intentionally small and stateless where possible. All cross-cutting state (authenticated user, active room, socket reference) is lifted into Context providers and consumed via custom hooks (useAuth, useSocket, useRoom).

4.1.3 State Management Layer

React's built-in Context API combined with useReducer is sufficient for the scope of this project and avoids the need for an external state management library such as Redux. Three contexts are defined:

- AuthContext - stores the current user object (including email, firstName, lastName, profileSetup, color) and exposes login, signup, and logout actions. No token management is needed client-side as the JWT is handled via HTTP-only cookie.
- RoomContext - stores the list of contacts (sorted by last message time) and the currently active conversation; exposes search and delete-DM actions.
- SocketContext - stores the Socket.io client instance and exposes emit helpers and event listener registration.

All three contexts are initialised at the application root (App.jsx) so that every child component can access global state via the corresponding hook without prop-drilling.

4.1.4 Routing Layer

React Router v6 is used with a declarative <Routes> configuration in App.jsx. Routes are divided into two groups:

- Public routes (/login, /signup) - accessible to unauthenticated users only; authenticated users are redirected to the chat view.
- Protected routes (/chat, /profile) - wrapped in the ProtectedRoute component; unauthenticated users are redirected to /login.

4.1.5 Service / API Layer

All communication with the backend is encapsulated in service modules under src/services/, keeping components free of network logic:

- authService.js - wraps POST /api/auth/signup, POST /api/auth/login, POST /api/auth/logout, GET /api/auth/userinfo, and POST /api/auth/update-profile. Authentication state is maintained server-side via an HTTP-only JWT cookie; no token handling is required in the frontend.
- contactService.js - wraps POST /api/contacts/search, GET /api/contacts/all-contacts, GET /api/contacts/get-contacts-for-list, and DELETE /api/contacts/delete-dm/:dmId.
- messageService.js - wraps GET /api/rooms/:id/messages to fetch conversation history on room entry.
- profileService.js - wraps PUT /api/users/profile to update first and last name.
- socketService.js - creates and exports the single Socket.io-client instance (configured with withCredentials: true and the ngrok-skip-browser-warning header); provides a sendMessage helper that emits the sendMessage event and registers the receiveMessage listener that pushes incoming messages into the active conversation state.

An Axios instance configured with the backend base URL and a request interceptor (to attach the JWT Authorization header) is shared across all HTTP service modules

4.1.6 Data Flow - Real-Time Messaging

The following sequence describes how a message is sent and received:

- User types in MessageInput and clicks Send (or presses Enter).
- MessageInput calls socketService.sendMessage(roomId, text).
- The Socket.io client emits a sendMessage event to the backend over the WebSocket connection.
- The backend delivers the message directly to the recipient user via a receiveMessage event.
- socketService's registered receiveMessage listener receives the message object (containing sender, recipient, content, messageType, and timestamp) and dispatches it to the active conversation's message array in the contact context state.
- ChatWindow, which is subscribed to the room's message array via useRoom, re-renders and scrolls to the bottom using a useEffect with a ref on the last message element.

4.1.7 Cloud / Hosting

The front-end static bundle (HTML, CSS, JS) is suitable for deployment on any static hosting platform such as Vercel or GitHub Pages, as it produces no server-side artefacts. For this project, the application runs locally on the developer's machine and communicates with the TA backend over localhost during development. No PaaS or SaaS services are used.

4.1.8 Front-end Architecture Diagram

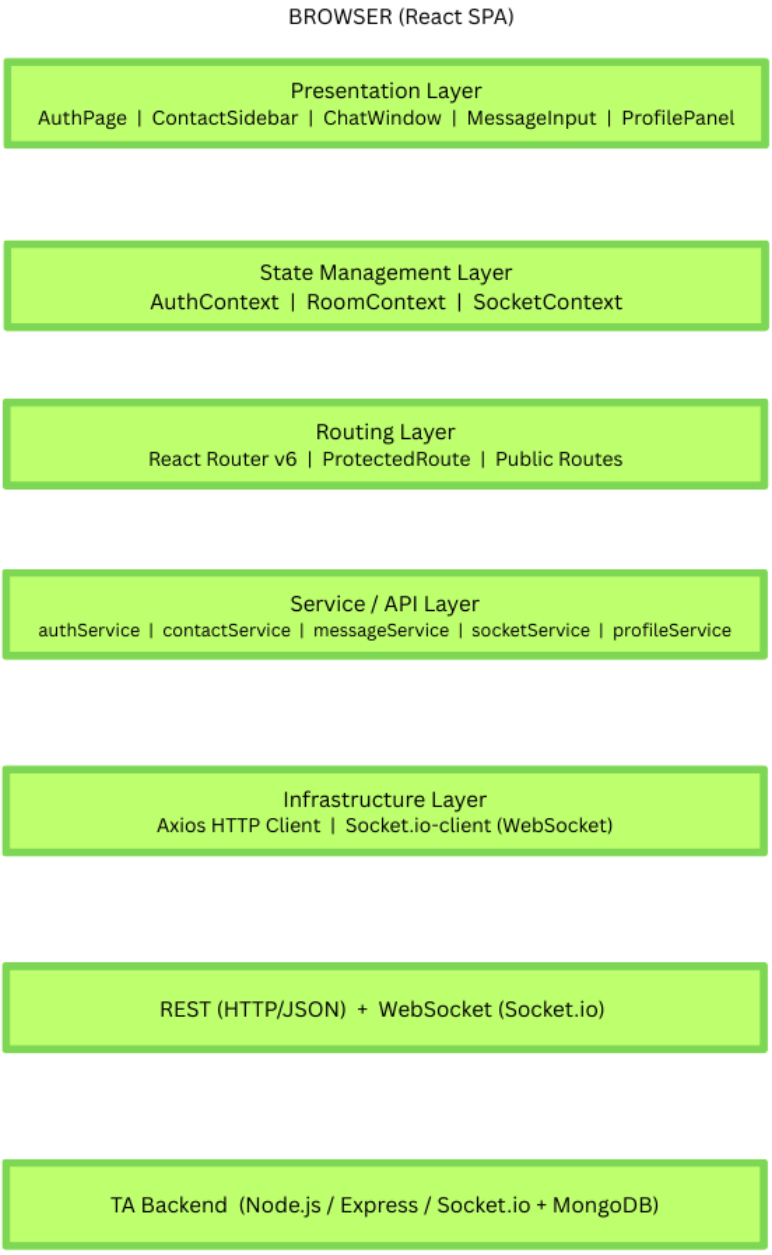

Figure 4.1 - Front-end architecture as a layered component model

4.2 Back-end Microservices Architecture

The backend is a Node.js/Express application that exposes RESTful HTTP endpoints and a Socket.io WebSocket server. For the scope of this project it operates as a monolithic service rather than a true microservices deployment.Its internal structure loosely separates concerns into three functional domains:

- Auth Service - handles signup and login with email/password, issues an HTTP-only JWT cookie, and exposes userinfo and update-profile endpoints.
- Contact Service - manages contact search, listing, and DM deletion.
- Message Service - stores and retrieves messages; broadcasts real-time events via Socket.io.

All three domains share a single MongoDB database. The frontend communicates with the backend through a single base URL; no API gateway or service mesh is required at this project's scale.

No PaaS or IaaS services are used by the TA backend in the development environment; it runs on localhost:5000 alongside the frontend development server.

5 Database Design

The database is MongoDB, a document-oriented NoSQL store. MongoDB was chosen for this project because its schema-flexible documents map naturally to JSON, which is the data format used throughout the MERN stack. It also allows the message collection to grow without requiring schema migrations as the application evolves.

The front-end's service layer must understand the data shape returned by the API in order to correctly render it. The three collections used by the application are:

Collection	Key Fields	Purpose
User	_id, email, passwordHash, firstName, lastName, image, color, profileSetup, createdAt	Stores user credentials and profile data
Rooms	_id, name, createdBy, createdAt, members[]	Tracks chat room metadata and DM membership
Messages	_id, roomId, senderId, senderName, text, timestamp	Persists all chat messages per room

The users collection stores credentials and profile data. The rooms collection maintains a members array of user IDs so that the backend can enforce room access and the front-end can display membership. The messages collection references both roomId and senderId as foreign keys; however, since MongoDB does not enforce referential integrity, the senderName is denormalised into each message document so that chat history can be displayed without a join, improving read performance in the ChatWindow.

The frontend is tightly coupled to the shape of the TA-provided REST and WebSocket API. Any undocumented changes to endpoint paths, payload structure, or Socket.io event names will break frontend functionality without any compile-time warning.

6. Issue and Concerns

- Socket.io's built-in reconnection handles transient network drops, but messages emitted by other users while the client is disconnected are not replayed.
- This is the first time I try to build a Front-end website.
- The 11-week project timeline limits the depth of testing, accessibility work, and performance optimisation that can be realistically completed. Unit tests with React Testing Library and end-to-end tests with Cypress are aspirational; the project will prioritise functional correctness and sprint deliverables over comprehensive test coverage.